S

PRODUCT ROADMAP · FEATURE IDEAS

# SmartCart: what to build next

A detailed, prioritised plan to take SmartCart from a secure storefront with a cart to a complete, production-ready e-commerce platform: checkout, payments, an admin panel, a richer shopping experience, and a path to scale.

Spring Boot 4 | Angular 22 | PostgreSQL | JWT + OAuth2 | Docker

| PROJECT | VERSION | DATE |
| --- | --- | --- |
| SmartCart e-commerce | Roadmap v1.0 | October 2026 |

CONTENTS

# What's inside


1 Executive summary

2 Where SmartCart stands today

3 Phase 0: Foundations & hardening

4 Phase 1: Checkout & orders

5 Phase 2: Payments

6 Phase 3: Admin & inventory

7 Phase 4: Customer experience

8 Phase 5: Identity & security

9 Phase 6: Performance & scale

10 Phase 7: DevOps & deployment

11 Innovation ideas (AI & more)

12 Prioritisation matrix

13 Timeline

14 Success metrics & learning resources


## How to read each idea

### Every idea answers four questions

- **Why:** the user or business value
- **How:** the technical approach in this codebase
- **Done when:** a checkable finish line
- **Chips:** effort and priority

### Effort scale (one developer)

| | |
|---|---|
| S | 1–2 days |
| M | 3–5 days |
| L | 1–2 weeks |
| XL | 3+ weeks |

Priority: **Now** needed before going live · **Next** high value, right after · **Later** nice to have or for scale.

1 · EXECUTIVE SUMMARY

# From storefront to platform

SmartCart already has the hardest-to-retrofit parts done properly: a secure identity layer (email and Google sign-in, RS256 JWTs, instant revocation), a server-side cart with stock checks, an audit log, a modern responsive UI with dark mode, and a Docker Compose setup. What's missing is the part that makes it a shop: **customers can't pay yet**. This roadmap closes that gap first, then adds the admin, experience and scale features in an order where every phase ships something usable.

| 8 | 40+ | ~6 |
|---|---|---|
| PHASES | CONCRETE IDEAS | MONTHS |

## Guiding principles

### 1. Ship the money path first

Checkout → payment → order management is the critical path. Everything else is secondary until a customer can buy something end to end.

### 2. Keep the modular monolith

Stay one deployable, but enforce module boundaries (user, product, cart, order, payment, inventory, audit) so services can be split out later without a rewrite.

### 3. Correctness over cleverness

Money and stock need transactions, idempotency and tests. Snapshot prices into orders and never trust the client with totals.

### 4. Every phase is demo-able

Each phase ends with a working feature, tests and an updated demo video ( `tools/demo-video` ).

## The phases at a glance

| # | PHASE | OUTCOME | EFFORT | PRIORITY |
|---|---|---|---|---|
| 0 | Foundations & hardening | Migrations, tests, CI, secrets rotated, rate limiting | L | Now |
| 1 | Checkout & orders | Cart becomes an order; stock is reserved safely | XL | Now |
| 2 | Payments | Stripe payments, webhooks, refunds | L | Now |
| 3 | Admin & inventory | Product CRUD, order management, stock ledger, dashboard | XL | Next |
| 4 | Customer experience | Filters, addresses, wishlist, reviews, coupons, Arabic/RTL | XL | Next |
| 5 | Identity & security | Email verification, password reset, refresh tokens, 2FA | L | Next |
| 6 | Performance & scale | Caching, CDN images, search, events, load tests | L | Later |
| 7 | DevOps & deployment | Cloud deploy, HTTPS, monitoring, backups | M–L | Next |

2 · BASELINE

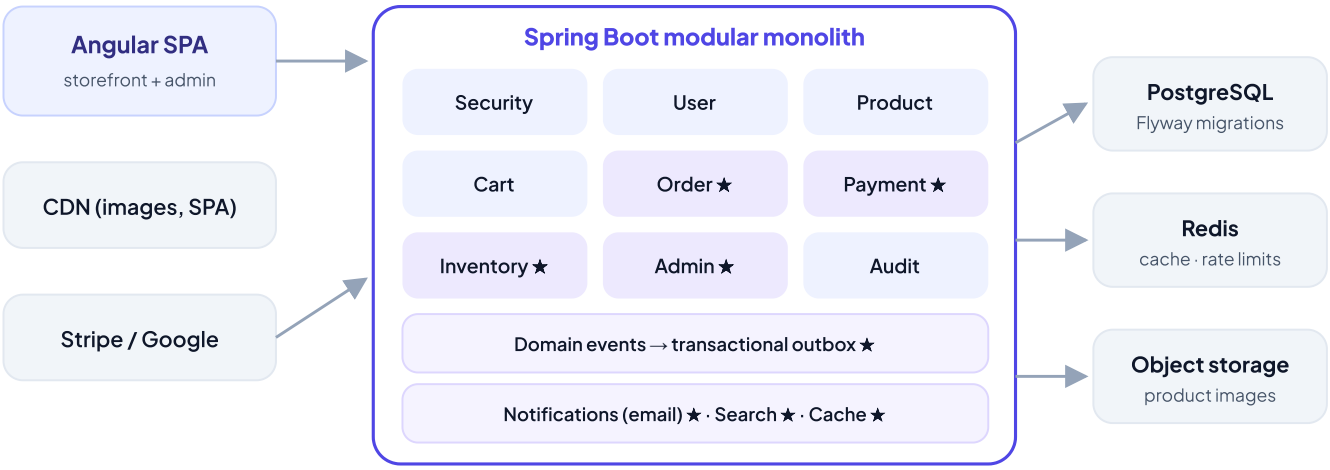
# Where SmartCart stands today

An honest inventory of the codebase as of October 2026, so every phase starts from facts.

| MODULE | STATUS | WHAT EXISTS | GAP |
|---|---|---|---|
| Security / user | Done | Email + Google sign-in, RS256 JWT, BCrypt, profile, password change, deactivate/reactivate/delete, roles (CUSTOMER, ADMIN), admin seeding | Email verification, password reset, refresh tokens, rate limiting |
| Product | Read-only | Public catalogue with search, pagination, detail, images (bytea + ETag), categories and brands | Admin CRUD, filters/sorting, variants |
| Cart | Done | Server-side cart, stock and per-line limits, currency guard, optimistic locking | Guest cart, saved-for-later |
| Order | Partial | Entities, full status enum, read-only order history | **No checkout**; legacy `saveOrderToCart` to remove |
| Payment | Empty | Package placeholder | Everything |
| Inventory | Empty | Stock is a column on `product` | Reservations, stock ledger, alerts |
| Audit | Done | Per-user activity log for account and cart events | Admin actions, order and payment events |
| Frontend | Done | Shop, product, cart, account, orders, activity, auth pages; dark mode; toasts | Checkout UI, admin UI |
| DevOps | Partial | Dockerfiles + Compose (db, backend, frontend), demo-video recorder | CI/CD, cloud deploy, monitoring |

**Known risks to fix first (Phase 0):** an old RSA private key in git history, a Google client secret possibly in history, `ddl-auto=update` instead of migrations, `debug=true` in `application.properties`, no rate limiting on login, and very few automated tests.

## Target architecture (end of roadmap)




★ = new in this roadmap.

## Module boundaries (target)

Each module owns its tables and exposes a small service API. Other modules may call that API, or listen to its events, but never touch its repositories. **Spring Modulith** (idea 0.7) can verify these rules in a test, so the monolith stays splittable.

| MODULE | OWNS | MAY DEPEND ON | PUBLISHES EVENTS |
|---|---|---|---|
| **user** | customer, roles, addresses | (none) | CustomerRegistered, CustomerDeleted |
| **security** | token issuing and verification | user | (none) |
| **product** | product, category, brand, images | (none) | ProductPriceChanged, ProductDeactivated |
| **inventory** | stock levels, stock_movement | product | StockLow, StockReplenished |
| **cart** | cart, cart_item | user, product, inventory (read) | (none) |
| **order** | orders, order_item, status history | user, cart, inventory | OrderPlaced, OrderCanceled, OrderCompleted |
| **payment** | payment, processed_webhook_event | order | PaymentSucceeded, PaymentFailed, RefundIssued |
| **notification** | email templates, outbox | listens to events only | (none) |
| **audit** | audit_log | listens to events only | (none) |
| **admin** | (no tables) | product, inventory, order, user | (none) |

> Notice that **notification** and **audit** depend on nothing; they only react to events. That's what lets them become separate services first, as described in 6.5.

# 0 Foundations & hardening

Do this before adding features. Every later phase is safer and faster because of it.

Effort: L (≈ 2 weeks) | Priority: Now | Depends on: nothing

## 0.1 Rotate leaked secrets and clean the config

S Now

**Why:** an RSA private key was committed in `29d697c` and is on GitHub; commit `966f65e` may contain the Google client secret. Deleting files doesn't remove them from history.

**How:** generate a new key pair (already git-ignored), create a new Google client secret and revoke the old one, then optionally rewrite history with `git filter-repo`. Remove `debug=true`. Split configuration into Spring profiles: `application-dev.properties` (show SQL, permissive CORS) and `application-prod.properties` (strict, everything from environment variables).

**Done when:** no secret in the current tree or history is still valid, and the prod profile starts only from environment variables.

## 0.2 Database migrations with Flyway

M Now

**Why:** `ddl-auto=update` never drops or relaxes columns. It already broke Google sign-up once and needed hand-written fix scripts ( `db/001…` , `db/002…` ). Checkout will add several tables; they must be versioned.

**How:** add `spring-boot-starter-flyway` , dump the current schema as `V1__baseline.sql` , set `spring.flyway.baseline-on-migrate=true` for existing databases and switch Hibernate to `ddl-auto=validate` . Move the dev seed into `R__dev_seed.sql` (repeatable), loaded only in the dev profile.

**Done when:** a fresh database is built entirely by Flyway and the app starts with `validate` .

## 0.3 An automated test safety net

L Now

**Why:** checkout and payments will touch money and stock; regressions there are costly. Today the backend has essentially one context-load test.

**How:** three layers:

- **Unit:** `CartServiceImpl` rules (merge quantities, stock and line limits, currency guard), `AuthServiceImpl` (enumeration-safe login, reactivation).
- **Web slice:** `@WebMvcTest` + `spring-security-test` for every rule in `SecurityConfig` : anonymous → 401, customer on an admin endpoint → 403, users only ever see their own data.
- **Integration:** `@SpringBootTest` + **Testcontainers PostgreSQL** for repositories and the end-to-end cart → order flow.

**Done when:** about 70% line coverage on the service layer, and every security rule has a test.

## 0.4 Continuous integration

S Now

**How:** a GitHub Actions workflow on every push and pull request: `./mvnw verify` on JDK 21, `npm ci && ng test && ng build` , a Docker image build, plus Dependabot and secret scanning. Pin the JDK to an LTS version (the JDK 27 / Lombok incident shows why).

**Done when:** the default branch is protected and pull requests can't merge while red.

### 0.5 Rate limiting and brute-force protection

M Now

**Why:** `/auth/login`, `/register` and `/reactivate` are public and currently unlimited, which invites credential-stuffing attacks.

**How:** Bucket4j filter keyed by IP and by email (for example 5 attempts per minute per email, 30 per IP), returning `429 Too Many Requests` with `Retry-After`. In-memory first, Redis-backed later (Phase 6). Add progressive account lockout using the existing `locked` flag, and record `SIGN_IN_FAILED` in the audit log.

**Done when:** a scripted brute-force run is throttled and visible in the audit log.

### 0.6 Observability basics

S Next

**How:** Spring Boot Actuator (`health`, `info`, `metrics`, exposed only internally); a request-ID filter that puts a correlation ID in the MDC and the `X-Request-Id` response header; JSON logs in prod (Spring Boot structured logging). Return the request ID in `ProblemDetail` so users can quote it to support.

**Done when:** any 500 error can be traced from the browser to a single log entry.

### 0.7 Enforce module boundaries

S Next

**How:** add **Spring Modulith** and a test that calls `ApplicationModules.of(ECommerceApplication.class).verify()`. It fails the build when, for example, the cart module reaches into the user module's repository. It can also generate C4/PlantUML module diagrams for the README.

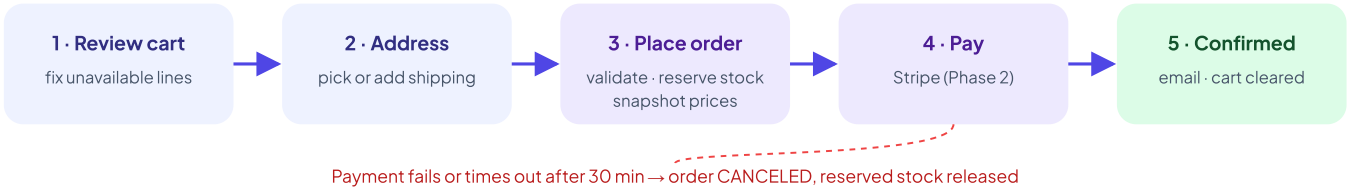
4 · PHASE 1

# 1 Checkout & orders

The critical path: turn a cart into an order, safely.

Effort: XL (≈ 3–4 weeks) | Priority: Now | Depends on: Phase 0 (migrations, tests)

## The checkout flow




### 1.1 Place an order from the cart

L Now

**How:** `POST /api/v1/orders` in one `@Transactional` method:

1. Load the caller's cart (user id from the token, never the body); reject it if empty or `hasUnavailableItems`.
2. For each line, **reserve stock atomically** (1.2).
3. Create the `Order` with status `PENDING_PAYMENT` and an `OrderItem` per line that **copies** product name, unit price and currency (the price snapshot). Later price changes must never alter past orders.
4. Compute totals on the server (subtotal, shipping, tax, discount), clear the cart and publish an `OrderPlaced` event.

Remove the legacy `OrderService.saveOrderToCart` and the `SAVED_TO_CART` status, which mixed the cart and order concepts.

**Done when:** an integration test places an order, stock drops, the cart is empty and a later price change leaves the order total unchanged.

### 1.2 Concurrency-safe stock reservation

M Now

**Why:** two customers buying the last unit at the same moment must not both succeed (overselling).

**How:** one conditional SQL update per line. The database guarantees only one of two concurrent buyers can take the last unit:

```
@Modifying
@Query("UPDATE Product p SET p.stockQuantity = p.stockQuantity - :qty "
     + "WHERE p.productId = :id AND p.stockQuantity >= :qty AND p.status = 'ACTIVE'")
int reserve(Integer id, int qty);   // 0 rows updated → INSUFFICIENT_STOCK, roll back the
whole order
```

Alternatively, add `@Version` to `Product` (optimistic locking) and retry on conflict. Release stock (`+ qty`) when an order is cancelled or its payment expires (a scheduled job every 5 minutes).

**Done when:** a test with 20 parallel threads buying the last 5 units produces exactly 5 successes and stock never goes negative.

### 1.3 Idempotent "Place order"

S Now

**How:** the SPA sends an `Idempotency-Key` header (a UUID created when the checkout page opens). The server stores it with the order, with a unique constraint per customer. A double click or network retry returns the *same* order instead of creating two.

### 1.4 Order lifecycle (state machine)

M Now

**How:** encode the allowed transitions in one place (an `OrderStatus.canTransitionTo(next)` method) and reject everything else with `409 INVALID_ORDER_TRANSITION`. Record each change in an `order_status_history` table and the audit log.

| FROM | ALLOWED NEXT STATES | WHO / TRIGGER |
|---|---|---|
| **PENDING_PAYMENT** | CONFIRMED, CANCELED | Payment webhook / timeout / customer |
| **CONFIRMED** | IN_PROGRESS, CANCELED, ON_HOLD | Admin; customer cancel within 1 hour |
| **IN_PROGRESS** | COMPLETED, ON_HOLD | Admin (packed / shipped / delivered) |
| **ON_HOLD** | IN_PROGRESS, CANCELED | Admin |
| **COMPLETED** | REQUEST_REFUND | Customer within 14 days |
| **REQUEST_REFUND** | CANCELED (refunded), COMPLETED (rejected) | Admin + payment refund |

### 1.5 Shipping addresses

M Now

**How:** a new `address` table (customer, name, line 1/2, city, region, postal code, country, phone, default flag) with `/api/v1/users/me/addresses` CRUD. At checkout, **copy** the chosen address into the order (as an `@Embeddable` shipping address), so editing the address book never changes past orders. Reuse the existing `@ValidPhoneNumber`.

### 1.6 Order pages and confirmation email

M Now

**How:** Angular checkout steps (review, address, confirm), an order detail page with a status timeline, and "cancel" / "request refund" buttons shown only when the transition is allowed. Send emails with `spring-boot-starter-mail` and Thymeleaf templates (order confirmation, shipped, refunded), triggered by domain events after the transaction commits ( `@TransactionalEventListener` ), so an email failure never rolls back an order. Use MailHog/Mailpit in dev.

## New API surface

| METHOD | PATH | PURPOSE |
|---|---|---|
| POST | `/api/v1/orders` | Place order from cart (Idempotency-Key) |
| GET | `/api/v1/orders/{id}` | Order detail: own orders only, otherwise 404 |
| POST | `/api/v1/orders/{id}/cancel` | Customer cancel (state machine permitting) |
| POST | `/api/v1/orders/{id}/refund-request` | Request a refund with a reason |
| GET/POST/PUT/DELETE | `/api/v1/users/me/addresses` | Address book |

5 · PHASE 2

# 2 Payments

Take real money, without ever touching card numbers.

Effort: L (≈ 2 weeks) | Priority: Now | Depends on: Phase 1

**Golden rule:** card data never reaches SmartCart's servers. With a hosted payment page (Stripe Checkout or Payment Element), the PCI DSS compliance burden drops to the lightest self-assessment questionnaire (SAQ A), instead of a full audit.

## 2.1 Stripe payment flow

L Now

**How:** after "Place order", the backend creates a Stripe **Checkout Session** (or PaymentIntent) for the server-computed total, stores its id on a new `payment` row (order, provider, provider reference, amount, currency, status) and returns the redirect URL. The customer pays on Stripe's page and comes back to `/checkout/success?order=…`. That page only *shows* a status: **the order is confirmed by the webhook, never by the browser redirect**, because a redirect can be faked.

**Done when:** paying with Stripe's test card `4242 4242 4242 4242` moves the order to CONFIRMED, and a declined test card leaves it PENDING_PAYMENT.

## 2.2 Secure, idempotent webhooks

M Now

**How:** `POST /api/v1/payments/webhook/stripe` is public but **verifies the `Stripe-Signature` header** against the webhook secret before reading anything. Store each processed event id in a `processed_webhook_event` table (unique key), because Stripe may deliver the same event more than once and it must be applied only once. Handle `checkout.session.completed`, `payment_intent.payment_failed` and `charge.refunded`. Test locally with `stripe listen --forward-to`.

## 2.3 Refunds and payment-provider abstraction

M Next

**How:** a `PaymentProvider` interface (`createPayment`, `refund`, `parseWebhook`) with a `StripePaymentProvider` implementation (Strategy pattern), so PayPal, Paymob (Egypt) or Tap/Telr (UAE) can be added without touching order code. Approving a refund calls `provider.refund`; the `charge.refunded` webhook then cancels the order and restocks the items.

## 2.4 Cash on delivery

S Next

**Why:** still a popular payment method in Egypt and much of the region. **How:** a `CodPaymentProvider` that confirms the order immediately with payment status `DUE`, marked `PAID` by an admin on delivery. A good proof that the abstraction in 2.3 works.

## 2.5 Invoices

S Later

**How:** generate a PDF invoice per confirmed order (OpenPDF or Thymeleaf + Flying Saucer) with a sequential invoice number, downloadable from the order page and attached to the confirmation email.

# 3 Admin & inventory

Let the shop owner run the shop without touching the database.

Effort: XL (≈ 4 weeks) | Priority: Next | Depends on: Phases 1–2

> The foundations already exist: the `ROLE_ADMIN` role, `AdminSeeder` and `@EnableMethodSecurity`. Admin endpoints live under `/api/v1/admin/**`, protected by a URL rule *and* `@PreAuthorize` on each service method.

## 3.1 Product management (CRUD + images)

L · Next

**How:** create, edit, deactivate (soft delete: orders reference products) and duplicate products; manage categories and brands. Image upload via `multipart/form-data`, **validating the real file type by its magic bytes, not the extension**, capping the size (for example 2 MB), re-encoding with ImageIO to strip metadata, and storing several sizes (thumbnail, card, full). Every change is audited.

## 3.2 Inventory module: a stock ledger

L · Next

**Why:** today stock is a single number, so you can't tell *why* it changed. **How:** fill the empty `inventory` module with a `stock_movement` table (product, delta, reason: `RESTOCK | SALE | RESERVATION_RELEASED | RETURN | ADJUSTMENT`, reference, actor, timestamp). Every stock change goes through `InventoryService`, which writes the movement and the new quantity in one transaction. The current stock is always explainable as the sum of the movements, the same principle accountants use.

Add low-stock thresholds with alerts, and bulk restock via CSV with a dry-run preview.

## 3.3 Order management

M · Next

**How:** a filterable, paginated order list (status, date range, customer email, amount) and an order detail view with the allowed next statuses as buttons (from the state machine), internal notes, tracking number and carrier, refund approval and packing slip.

## 3.4 Dashboard and reports

M · Next

**How:** KPI tiles (revenue today / 7 days / 30 days, orders, average order value, conversion = orders ÷ carts), a revenue-by-day chart, top products, low-stock list and recent orders. Use aggregate SQL queries (`GROUP BY date_trunc('day', created_at)`), cached for 5 minutes, plus CSV export of orders.

## 3.5 Customer management

M · Later

**How:** search customers; view profile, orders and activity log; lock or unlock accounts (the existing `locked` flag already revokes tokens instantly through `JwtFilter`); grant or revoke the admin role. Every admin action is audited with the admin's id: *who did what to whom*.

## 3.6 Admin UI

L · Next

**How:** a lazy-loaded Angular feature area at `/admin` with its own layout (sidebar navigation), an `adminGuard` that checks the `ROLE_ADMIN` claim (UX only; the server enforces it), data tables with server-side sorting and paging, and reusing the existing theme, dark mode and toasts.

# 4 Customer experience

Features that make people find, trust and come back.

Effort: XL (pick and choose) | Priority: Next | Depends on: Phase 1

## 4.1 Catalogue filters and sorting

M Next

**How:** filter by category, brand, price range and in-stock only; sort by price, newest and best-selling. Build the query from optional criteria with JPA **Specifications** (or Querydsl) rather than ever-longer repository method names. Keep filters in the URL so results can be shared.

## 4.2 Reviews and ratings (verified purchase)

M Next

**How:** only customers with a COMPLETED order containing the product can review it, once per product. A 1–5 rating plus text, admin moderation (hide, with the reason audited), and the average rating with count stored on the product and updated in the same transaction. This blocks fake reviews by design.

## 4.3 Wishlist and price-drop alerts

S Next

**How:** a heart icon on cards and product pages, and `/api/v1/users/me/wishlist`. Stores the price at the moment of saving; a nightly job emails "price dropped" or "back in stock" notices (opt-in, with one-click unsubscribe).

## 4.4 Guest cart, merged on sign-in

M Next

**Why:** requiring sign-in before "Add to cart" loses customers. **How:** keep a guest cart in `localStorage` (product ids and quantities only, never prices). On sign-in, `POST /api/v1/cart/merge` adds each line through the existing `CartService.addItem`, so every stock and limit rule still applies, and reports any lines that couldn't be merged.

## 4.5 Coupons and promotions

L Later

**How:** a `coupon` table (code, percent or fixed amount, minimum order, valid from/to, total and per-customer usage limits, eligible categories), validated at checkout on the server, with redemptions recorded atomically to prevent over-use. The discount is snapshotted into the order like prices are.

## 4.6 Arabic language and right-to-left layout

L Next

**Why:** a natural fit for customers in Egypt and the Gulf. **How:** Angular i18n (or Transloco for runtime switching), `dir="rtl"` with Bootstrap's RTL stylesheet, CSS logical properties (`margin-inline-start` instead of `margin-left`), locale-aware number, currency and date formats (EGP, AED), Arabic product names and descriptions (a translation table per product), and an Arabic-capable font (for example IBM Plex Sans Arabic or Cairo).

## 4.7 Product variants (size, colour) with SKUs

XL Later

**How:** split `Product` into a product (shared description and images) and `product_variant` rows (SKU, attributes, price, stock). The cart and orders then reference variants. A big migration: decide early.

### 4.8 Small touches with big impact

S each Later

- **Recently viewed** products (local, no backend)
- **Order tracking** page with carrier link and a status timeline
- **Image gallery** with zoom and multiple images per product
- **Accessibility audit** (axe / Lighthouse), aiming for WCAG 2.2 AA
- **SEO:** Angular SSR or prerendering for product pages, product structured data (JSON-LD), sitemap

# 5 Identity & security

Close the remaining gaps from `docs/SECURITY.md` §7.

Effort: L (≈ 2–3 weeks) | Priority: Next | Depends on: Phase 0

## 5.1 Email verification for local sign-up

M Now

**Why:** Google accounts are linked by *verified* email, but local sign-ups are not verified at all. **How:** an `email_verified` column. On sign-up, send a link containing a random 32-byte token, storing only its **SHA-256 hash**, valid for 24 hours and usable once. Unverified users can browse and fill a cart, but checkout requires verification.

## 5.2 Forgot / reset password

M Now

**How:** `POST /auth/forgot-password` **always** returns 202 (never reveals whether the email exists). The email contains a single-use hashed token valid for 30 minutes. A successful reset revokes all existing sessions (a `tokens_valid_after` timestamp checked in `JwtFilter`) and is recorded as `PASSWORD_RESET` in the audit log, with a notification email.

## 5.3 Refresh tokens with rotation

L Next

**How:** a short access token (10 minutes) plus a long refresh token (14 days) stored as an **HttpOnly, Secure, SameSite=Strict cookie**, with a hash kept in a `refresh_token` table (family id, device, expiry, revoked). Each refresh issues a new refresh token and revokes the old one. Reuse of a revoked token means it was stolen, so the whole token family is revoked (the reuse detection described in RFC 9700). The SPA then keeps the access token in memory instead of `localStorage`. Because a cookie is now involved, CSRF protection is re-enabled for `/auth/refresh`.

## 5.4 Active sessions and "sign out everywhere"

S Next

**How:** built on 5.3: list devices (browser, approximate location, last used) on the account page, with per-session and global sign-out. Pairs naturally with the existing activity page, plus "new sign-in from a new device" emails.

## 5.5 Two-factor authentication (TOTP)

M Later

**How:** authenticator-app codes (RFC 6238) with a QR enrollment page, an encrypted secret at rest, 10 one-time recovery codes (hashed), and a two-step login (password → short-lived "mfa_pending" token → code → real token). Required for admins, optional for customers. Passkeys (WebAuthn) are a strong follow-up.

## 5.6 More providers, key rotation and security headers

M Later

- **More sign-in providers:** Microsoft, GitHub or Apple. The `AuthProvider` enum and success handler are already provider-agnostic; only GitHub needs extra work, because it doesn't return `email_verified` the same way.
- **Key rotation:** a `kid` header plus a public `/.well-known/jwks.json` endpoint with current and previous public keys, enabling zero-downtime rotation and verification by future services.
- **Security headers:** Content-Security-Policy, HSTS, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`.
- **Dependency scanning:** OWASP Dependency-Check or Snyk in CI.

# 6 Performance & scale

Meet the README targets (API < 500 ms, page load < 2 s) as traffic grows.

Effort: L | Priority: Later | Depends on: Phase 0 (metrics), Phase 7 (deploy)

## 6.1 Measure first: load tests

S Next

**How:** k6 or Gatling scripts for browse → search → add to cart → checkout, run in CI against a staging environment, with thresholds (p95 < 500 ms, error rate < 1%). Find bottlenecks with Actuator metrics and SQL logging before optimising anything.

## 6.2 Caching

M Later

**How:** start with an in-memory Caffeine cache for `@Cacheable` catalogue reads (categories, brands, product pages), evicted on admin updates. Move to **Redis** once there is more than one backend instance; it can also back rate limiting (0.5) and sessions. Never cache anything per-user, such as carts or profiles.

## 6.3 Images to object storage and a CDN

M Later

**Why:** images are stored in PostgreSQL as `bytea` and served by the API. That works for 24 products but not for thousands. **How:** S3-compatible storage (AWS S3, Cloudflare R2 or MinIO locally); admin uploads use pre-signed URLs; images are served from a CDN in WebP/AVIF at several sizes; `Product` stores only the image key. Provide a one-off migration from the `product_image` table.

## 6.4 Better search

M Later

**How:** step 1 is PostgreSQL full-text search (a `tsvector` column with a GIN index, ranking by relevance, plus `pg_trgm` for typo tolerance), which needs no new infrastructure. Step 2, only if needed, is OpenSearch or Meilisearch fed by product events, adding autocomplete and facets.

## 6.5 Events and the path to microservices

L Later

**How:** modules communicate through domain events ( `OrderPlaced` , `PaymentSucceeded` , `StockLow` ). Spring Modulith's event publication registry provides a **transactional outbox**: events are saved in the same transaction as the business change and delivered reliably afterwards. When a module needs to scale independently (likely notifications or search first), externalise its events to Kafka or RabbitMQ and extract it as a service. This realises the Kafka saga sketched in the README's sequence diagram, without a big-bang rewrite.

## 6.6 Query hygiene

S Next

- Add the indexes the README already promises: `product(product_name)` , `product(category_id)` , `orders(customer_id, created_at)` , `audit_log(customer_id, created_at)`
- Use keyset pagination for the activity log and admin order lists (fast at any depth)
- Run a Hibernate statistics test that fails if a request issues more than N queries (catches N+1 regressions)
- Set an explicit HikariCP pool size and timeouts

# 7 DevOps & deployment

From Docker Compose on a laptop to a public, monitored HTTPS site.

Effort: M–L | Priority: Next (deploy early, deploy often) | Depends on: Phase 0

## 7.1 Continuous delivery pipeline

M Next

**How:** extend the CI from 0.4: on merge to `master`, build versioned images (Git SHA tag), push them to GitHub Container Registry, deploy to **staging** automatically, run smoke tests plus the demo-video script as an end-to-end check, then promote to **production** with a manual approval. Flyway runs migrations on startup.

## 7.2 Hosting options

M Next

| OPTION | GOOD FOR | NOTES |
|---|---|---|
| **Render / Railway / Fly.io** | Portfolio and first customers | Deploys straight from Docker; managed Postgres; HTTPS included; lowest operational effort |
| **A single VPS + Docker Compose + Caddy** | Learning, low cost | Caddy handles automatic HTTPS; you manage backups and updates yourself |
| **AWS (ECS Fargate + RDS + S3 + CloudFront) or Azure equivalents** | Real production and scale | The most to learn; strong CV value; infrastructure as code with Terraform |

## 7.3 Production readiness checklist

M Now (before go-live)

- **HTTPS everywhere** with HSTS; update the Google OAuth redirect URI and CORS origin to the real domain
- **Secrets** in the platform's secret store, never in images; JWT keys mounted as files
- **Automated daily database backups** with a tested restore (a backup you've never restored isn't a backup)
- **Monitoring:** uptime check, Prometheus + Grafana (or the platform's built-in tooling), and error tracking with Sentry (backend and Angular)
- **Alerts** on error rate, p95 latency, failed payments and webhook failures
- **Legal:** privacy policy, terms and a cookie notice; GDPR-style data export and the existing account deletion

# Innovation ideas

Ideas that would make SmartCart stand out as a portfolio project, ordered roughly from simplest to most ambitious.

### "Frequently bought together"

Count product pairs that appear in the same orders (a nightly SQL job) and show the top 3 on product and cart pages. Simple, explainable and effective; no machine learning needed to start.

M

### Abandoned-cart reminders

A carts table with `updated_at` is already there. Email customers whose cart has been idle for 24 hours (opt-in, at most one reminder), and track recovered revenue on the dashboard.

S

### Installable PWA

Angular service worker and a web manifest: installable on phones, cached catalogue pages, offline fallback, and later web push for order status updates.

M

### Loyalty points and gift cards

Earn points per completed order (recorded in a ledger table, like inventory), redeem them at checkout; gift cards as prepaid codes. Both reuse the coupon and ledger patterns from Phases 3–4.

L

### AI product descriptions for admins

In the admin product editor, "Generate description" sends the product name, attributes and image to an LLM API (for example Claude) and returns a draft in English and Arabic for the admin to edit. Always human-reviewed; the API key stays on the server.

M

### Semantic search

Store an embedding per product ( `pgvector` in the same PostgreSQL) so queries like "something to keep coffee warm on a hike" find the insulated bottle. Combine it with keyword search for the best of both.

L

### Shopping assistant chat

A chat panel backed by an LLM with **tool use**: it can call your own read-only APIs (search products, check stock, look up the caller's order status). It must never take actions on its own and must respect the same authorization as the user.

XL

### Experimentation

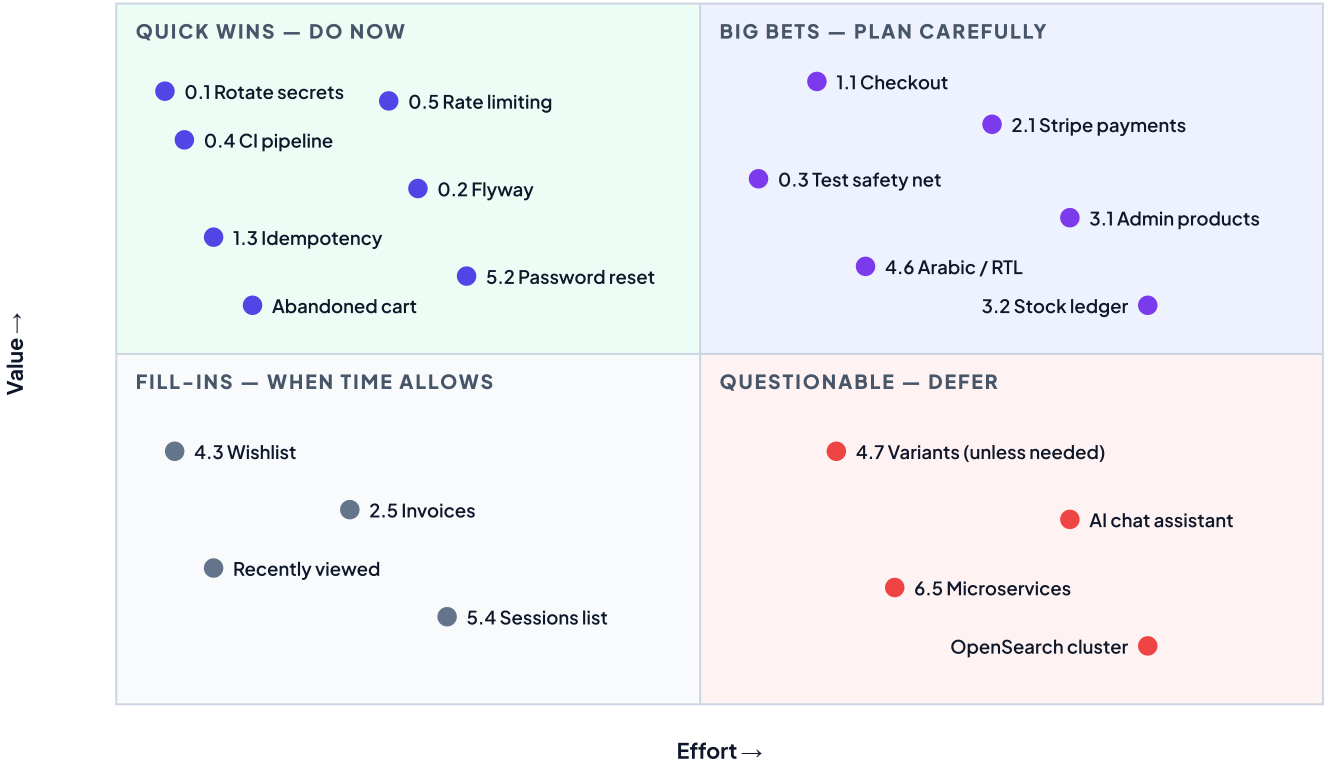
Feature flags (for example Unleash or a simple flags table) plus A/B tests on the hero, product card layout or checkout steps, measured by conversion on the dashboard.

M

**AI features checklist:** keep API keys server-side; rate-limit per user; never send personal data the feature doesn't need; show AI output as a draft or suggestion; log prompts and outputs for review; give users a non-AI fallback.

# Value vs effort

Each idea is placed by its user or business value and its effort. Start top-left (high value, low effort), schedule top-right deliberately, and leave the bottom-right for last.

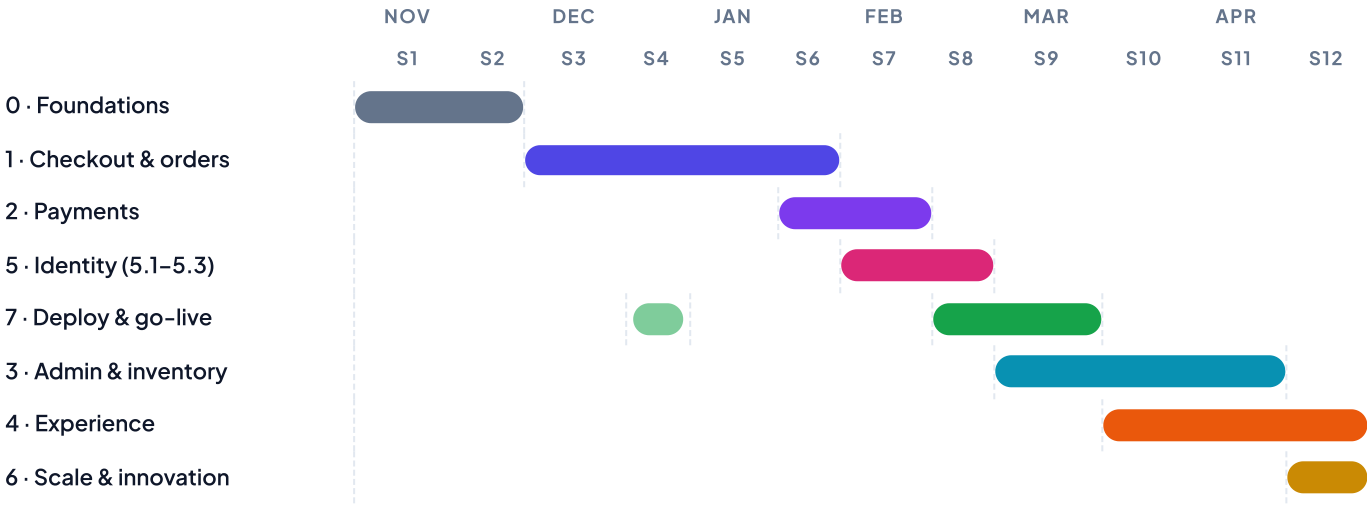



| RANK | IDEA | WHY NOW |
|---|---|---|
| 1 | 0.1 Rotate secrets · 0.2 Flyway · 0.4 CI | Cheap, and they remove risks that grow with every commit |
| 2 | 0.3 Tests · 0.5 Rate limiting | Safety net needed before touching money |
| 3 | 1.1–1.6 Checkout & orders | The one missing piece of the core shopping flow |
| 4 | 2.1–2.2 Stripe + webhooks | Turns orders into revenue |
| 5 | 5.1–5.2 Verify email · Password reset | Expected by every real user; required before launch |
| 6 | 7.1–7.3 Deploy + production checklist | Go live on a real domain |
| 7 | 3.x Admin & inventory | The owner can run the shop without SQL |
| 8 | 4.x Experience, then 6.x scale and innovation | Growth and differentiation |

13 · TIMELINE

# A six-month plan

Two-week sprints for one developer, starting November 2026. Phases overlap where they don't depend on each other. Treat the dates as a guide and re-plan at the end of every sprint.




The faded bar in S4 is an early staging deployment: deploy early so infrastructure problems surface before launch.

## Milestones

| WHEN | MILESTONE | DEMO |
|---|---|---|
| End of S2 (Nov) | **M0, Solid ground:** migrations, CI green, secrets rotated, rate limiting | Pull request blocked by a failing test |
| End of S6 (Jan) | **M1, First order:** checkout with stock reservation and order emails | Place an order; stock drops; email arrives |
| End of S7 (Feb) | **M2, First payment:** Stripe test mode end to end | Pay with a test card; webhook confirms the order |
| End of S9 (Mar) | **M3, Go-live (v1.0):** production deploy, verification, password reset | Public HTTPS URL; new demo video |
| End of S11 (Apr) | **M4, Self-service shop:** admin panel and inventory | Add a product, fulfil an order without SQL |
| End of S12 (Apr) | **M5, v2.0:** filters, reviews, Arabic, first innovation feature | Arabic RTL storefront walkthrough |

# Success metrics

### Product metrics

- **Checkout conversion:** orders ÷ carts with items (target: track a baseline first, then improve)
- **Order success rate:** confirmed ÷ placed orders > 95%
- **Cart abandonment** and recovery from reminders
- **Average order value** and repeat-customer rate

### Engineering metrics

- **API latency:** p95 < 500 ms (README goal)
- **Error rate:** < 0.5% of requests return a 5xx
- **Zero oversold orders** (stock ledger audit)
- **Test coverage:** ≥ 70% services; 100% of security rules
- **Deploy frequency** and time to restore after an incident

**Next step:** turn Phase 0 into GitHub issues (one per idea, with the "Done when" line as the acceptance criteria), group them into a "Sprint 1" milestone, and start with 0.1, rotating the leaked secrets.

## Learning resources by phase

| TOPIC | RESOURCE |
|---|---|
| Flyway migrations | Spring Boot reference, "Database Initialization" · documentation.red-gate.com/fd |
| Testing | Spring Boot reference, "Testing" · testcontainers.com/guides |
| Module boundaries | Spring Modulith reference: docs.spring.io/spring-modulith |
| Rate limiting | Bucket4j documentation: bucket4j.com |
| Concurrency & locking | Vlad Mihalcea, *High-Performance Java Persistence* · PostgreSQL docs, "Concurrency Control" |
| Payments | Stripe docs: Checkout, Webhooks (signature verification), Testing · PCI DSS SAQ A overview |
| Identity | OWASP cheat sheets: Forgot Password, Multifactor Authentication · RFC 9700 (OAuth 2.0 security) · RFC 6238 (TOTP) |
| Event-driven design | microservices.io, "Transactional outbox" and "Saga" patterns (Chris Richardson) |
| Domain modelling | Vaughn Vernon, *Domain-Driven Design Distilled* · Eric Evans, *Domain-Driven Design* |
| Internationalisation | Angular docs, "Internationalization" · Bootstrap docs, "RTL" |
| Deployment & operations | The Twelve-Factor App (12factor.net) · Google SRE book (sre.google/books) |